MPFT Study Guide (for the viva)

Read this top to bottom once, then use Parts 8-10 to quiz each other. Deeper detail lives in `PROTOCOL.md`, `ARCHITECTURE.md`, `TEST_REPORT.md`, `VIVA_PREP.md` (43 Q&A) and `WEB.md`.

---

1. The project in 30 seconds

> A normal file transfer uses **one TCP connection = one route**. If that route breaks, the transfer dies; if the file is damaged we find out at the end; if it is interrupted we start from zero. **MPFT** cuts the file into chunks and sends them over **several TCP connections ("paths") in parallel**. Every chunk is verified with **SHA-256**. If a path dies, its chunk goes back to a shared queue and another path sends it. If a chunk is corrupted, the receiver says **NACK** and it is resent. An interrupted transfer **resumes** by re-hashing what is already on disk. The "network" is emulated by a Python TCP proxy that adds delay, rate limits, corruption and path failures.

One-line answer to "what did you build?" A custom application-layer protocol over TCP with chunking, work-stealing multipath scheduling, integrity checking, recovery and resume, plus a network emulator, experiments, a Wireshark dissector and a live web UI.

---

2. Mental model (draw this on the whiteboard)

```
file --pread--> SENDER ---- path 0 ----[emulator: delay/rate/kill/corrupt]----> RECEIVER --pwrite--> file.part
                shared   ---- path 1 ----[emulator]---------------------------> verify chunk SHA-256   |
                queue    ---- path N ----[emulator]---------------------------> whole-file SHA-256 OK -> renam
                <-------------------- ACK / NACK / DONE ----------------------
```

Four ideas carry everything:

1. **Chunking** - file = N chunks of `chunk_size` (default 1 MiB). Chunk `i` is bytes `[i*cs, (i+1)*cs)`.
2. **Work stealing** - one shared queue of chunk indexes. Each path thread takes the next index only after its previous chunk was ACKed. Fast path returns sooner so it carries more. No bandwidth measurement needed.
3. **Layered integrity** - TCP checksum (network) -> header CRC-32 (framing) -> per-chunk SHA-256 (data, selective resend) -> whole-file SHA-256 (final proof, then rename `.part`).
4. **Recovery by requeue** - any NACK, timeout, reset or bad frame puts the chunk back in the queue (front, priority 0) with exponential backoff, max 5 retries per chunk.

---

3. Protocol cheat sheet

Header, 20 bytes, big-endian: `magic "MP"(2) | version(1) | type(1) | session_id(4) | chunk_index(4) | payload_len(4) | crc32 of first 16 bytes(4)`. In code: `struct.Struct(">2sBBIII")` + 4 CRC bytes (`mpft/protocol.py`).

#	Type	Direction	Payload
1	HELLO	S -> R	none, first message on every connection (carries session id)
2	MANIFEST	S -> R	JSON: name, size, chunk_size, total_chunks, file_sha256, chunk_sha256[]
3	MANIFEST_ACK	R -> S	JSON list of chunk indexes receiver already holds (resume)
4	CHUNK	S -> R	raw bytes
5	ACK	R -> S	chunk stored AND verified
6	NACK	R -> S	hash failed, resend
7	DONE	both	S: "all ACKed, verify". R: "verified and renamed"
8	ERROR	R -> S	reason text

Transfer sequence: HELLO -> MANIFEST -> MANIFEST_ACK -> (CHUNK -> ACK/NACK)* on all paths -> DONE -> DONE. The first path to connect sends the MANIFEST (lock in `ensure_manifest`); other paths wait for it.

Reject rules (connection closed, treated as path failure): bad magic, bad version, unknown type, header CRC mismatch, `payload_len > 64 MiB` (stops a corrupt length from making us allocate gigabytes).

Numbers: 5 retries per chunk (6th fails the transfer); chunk backoff `min(0.05*2^(n-1), 1.0)` s; path reconnect backoff `min(0.1*2^fails, 2)` s; path DOWN after 6 consecutive failures; 4 connect tries (0.2/0.4/0.8/1.6 s); chunk timeout 5 s.

---

4. Code map (who does what)

File	Lines	Key things to be able to point at
`mpft/protocol.py`	95	`pack_header`/`unpack_header` (CRC check), `recv_exact` (loops until n bytes - TCP is a stream), `build_manifest` (one streaming pass, hashes each chunk + whole file)
`mpft/sender.py`	264	`Sender.worker` (per-path thread loop), `send_chunk`, `on_ack`, `fail_path`, `requeue` (priority 0 + backoff + retry limit), `ensure_manifest`, `run`, `run_transfer` (the public API)
`mpft/receiver.py`	183	`Session` (holds `held` set, `.part` file, `_scan` for resume, `finalize` for whole-file hash + rename), `ReceiverHandle._serve` (per-connection loop), `_store` (verify, `pwrite`, ACK/NACK, duplicates)
`mpft/emulator.py`	162	`PathProxy`: `_reader` stamps due-time, `_writer` sleeps until due (delay, jitter, rate), `kill`, `restore`, corruption per 16 KiB block
`mpft/status.py`	50	writes `status/<role>-<session>.json` atomically once a second
`mpft/events.py`	50	thread-safe event bus used by sender/receiver/emulator for the live UI
`mpft/cli.py`	107	`send`, `receive`, `emulate`, `dashboard`, `web`, `rendezvous`
`web/*`	~1700	`node.py` event hub, `ui.py` local UI server (127.0.0.1), `peer.py` network-facing server, `ids.py`/`codes.py` receiver ID + pairing code, `relay.py` shares events between the two machines
`experiments/`		`run_experiments.py` (E1-E7) -> `results/*.csv`, `plot_results.py` -> PNGs
`wireshark/mpft.lua`		dissector for the MPFT header

File	Lines	Key things to be able to point at
`tests/`		46+ tests: protocol, transfer, failure, resume, emulator, web, Playwright e2e

Design constraints you can brag about: core is stdlib only; files are ~100-260 lines; file is never fully in RAM (sender `os.pread` one chunk at a time, receiver `os.pwrite` at `index*chunk_size`); at most one chunk per path is in memory.

---

5. Walkthrough: what happens, step by step

Normal transfer

1. `build_manifest` streams the file once: SHA-256 per chunk and for the whole file.
2. Sender starts one thread per target. Each connects, sends HELLO.
3. First connected path sends MANIFEST. Receiver opens `name.part` (preallocated), scans it for resume, replies MANIFEST_ACK with held indexes. Sender queues only the missing indexes.
4. Each worker: pop index -> `pread` -> CHUNK -> wait reply. ACK -> record bytes and RTT, loop.
5. Receiver `_store`: check index and length, SHA-256 vs manifest, `pwrite` at offset, add to `held`, ACK.
6. When `held` is full the receiver hashes the whole `.part`; sender sends DONE after all ACKs; receiver renames only if hash matches, replies DONE (or ERROR).

Path dies mid-transfer

Worker's send/recv raises (reset or timeout) -> `fail_path` records failure time -> `requeue(idx)` puts the chunk back as `(0, idx)` -> worker closes socket, backs off, tries to reconnect. Other paths pick the chunk up. Recovery time = first failure detected -> every in-flight chunk of failed paths ACKed. Measured: **0.15 s**.

Corruption

Receiver SHA-256 mismatch -> NACK, nothing written -> sender requeues. Corrupted data can never reach the output file; the whole-file hash is a second barrier.

Resume

New run, same file: receiver finds `name.part` of right size, **re-hashes each chunk on disk** vs the manifest, reports matches in MANIFEST_ACK. Damaged chunks fail the hash and are just resent. No separate state file -> nothing to get out of sync after a crash.

---

6. Results to memorise (from `results/*.csv`, 50 MB file, 256 KiB chunks, 40 Mbit/s + 10 ms per path)

Exp	What	Headline
E1	1/2/4/8 paths	**23 / 45.5 / 89 / 172 Mbit/s** - near-linear scaling

Exp	What	Headline
E2	paths 80/20/5 Mbit/s	byte share **74 / 20 / 6 %** - work stealing adapts
E3	one path 0-200 ms extra delay	time 7.8 -> 16 s, slow path share 58% -> 16%
E4	1 of 3 paths dies at 1.5 s	still succeeds, hash OK, **recovery 0.15 s**, +2.3 s total (lost capacity)
E5	0 / 1 / 5 % chunk corruption	0 / 2 / 12 retransmits of 200 chunks, always succeeds
E6	chunk 64K/256K/1M/4M	50 / 90 / 107 / 107 Mbit/s
E7	kill sender at 40%, rerun	resumed run skipped 80 chunks, 5.7 s vs 9.2 s

Why one 40 Mbit/s path gives only 23? Stop-and-wait: one chunk in flight per path, so each chunk costs transmit time + a round trip. More paths overlap these waits. A sliding window would fix it (future work).

Retry priority story (good to tell): with a plain FIFO, a retried chunk waited behind all fresh chunks -> recovery 6.9 s. Switching to `PriorityQueue` with `(0, idx)` for retries -> 0.22 s.

---

7. Web layer in one page

- `python -m mpft web ...` starts per user: a **UI server (127.0.0.1 only**, Host/Origin checked) and a **peer server (0.0.0.0)** exposing only `/health`, `/addresses`, `/offer`, `/events/{sid}`.
- **Receiver ID** = base32 of `host:port` (the machine's LAN address), so no server is needed. **Pairing code** = 6 random chars via a rendezvous service; expires in 10 min, single-use, rate limited.
- **Offer flow:** sender POSTs an offer; an ID offer waits for the receiver to click Accept (up to 120 s); a valid code auto-accepts. Accept returns a session token needed for `/events`.
- **Live dashboard:** events bus -> SSE -> JS reducer -> panels (chunk map, paths, throughput graphs, incidents, event log, trace). Reload or History **replays** the saved `.jsonl` through the same reducer.
- **Two-sided view:** sender relays its events to the receiver; clock offset = `((t1−t0)+(t2−t3))/2` (NTP-style).
- **Kill path / corrupt buttons** control the emulator for real TCP failures.

---

8. Likely viva questions (short answers) - quiz each other

Concepts 1. *Why TCP per path, not UDP?* Reliable ordered bytes, congestion control, NAT/firewall friendly; we add only chunk-level logic. 2. *Why header CRC if TCP has a checksum?* Protects framing (a wrong length desyncs the stream) and catches corruption injected above TCP by our emulator. 3. *Why per-chunk AND whole-file hash?* Chunk: pinpoint resend + trust for resume. File: final proof before rename. 4. *Work stealing vs round robin?* Round robin = slowest path finishes last, dead path blocks its share. Stealing adapts automatically. 5. *Why application-level ACK?* TCP ACK means "bytes arrived in kernel", ours means "stored and verified". 6. *Why is DONE two-way?* Sender needs a definite point confirming verification and rename. 7. *Why are duplicates harmless?* Already in `held` -> re-ACK, no rewrite. A lost ACK causes no damage. 8. *Why `pread/pwrite`?* Thread-safe positional I/O, out-of-order chunks, no whole file in RAM. 9. *Why thread per path, not asyncio?* Simple to explain; load is socket-

wait so the GIL is not the bottleneck (172 Mbit/s at 8 paths). 10. *How is delay emulated without reordering?* Reader stamps `due = max(prev_due, now+delay+jitter)`, writer sends at due time; due never decreases.

Failure scenarios 11. *Path dies?* See Part 5. Chunk requeued at front, others finish, path may reconnect, DOWN after 6 fails. 12. *All paths die?* Transfer fails with an error; `.part` kept so a rerun resumes. 13. *Chunk corrupted every time?* After 5 retries transfer fails (`corrupt_rate=1.0` test U12). 14. *Path goes silent (black hole)?* Chunk timeout 5 s, then same as dead path; detection takes up to the timeout (test U10). 15. *Sender crashes at 40%?* Rerun resumes; receiver re-hashes `.part`, skips verified chunks (E7). 16. *Manifest sent on a dead path?* Lock released on failure; another path sends it.

Honest limitations (say these before they ask) - No encryption or authentication; use only on a trusted network. Only path-traversal protection (base name only). - Stop-and-wait per path limits single-path throughput. - Emulator is on one machine, approximates real networks, does not drop IP packets (TCP hides loss anyway). - No run across two physical machines was done; Safari/Firefox untested. - `test_emulator` corruption test was flaky (~1 in 8 runs) - timing-dependent, not a protocol bug.

Improvements: sliding window per path, TLS/Noise, adaptive chunk size, true loss emulation, resume metadata that avoids re-hashing.

---

9. Tricky "gotcha" questions

- "*Your CRC covers only 16 bytes. Why not the payload?*" Payload is covered by SHA-256 which is stronger and per-chunk. CRC is only for framing.
- "*Is CRC-32 secure?*" No, it only detects accidental errors. SHA-256 provides integrity; neither provides authenticity.
- "*Does multipath always help?*" No. If paths share a bottleneck, aggregate does not grow. Gain needs independent routes (e.g. WiFi + hotspot).
- "*What if two paths deliver the same chunk?*" Possible after a requeue race; receiver detects via `held` and counts a duplicate.
- "*Why is goodput less than sum of link rates?*" Per-chunk round trip, headers, final tail on the slowest path, hashing cost.
- "*What is goodput vs throughput?*" Goodput = useful file bytes / total time (excludes headers and retransmits).
- "*Corrupt rate 5% but 12 of 200 retransmits (6%)?*" Random variation; each experiment ran once.
- "*How do you know the emulator is realistic?*" It is not claimed to be; it gives reproducible, controllable conditions.

---

10. Demo checklist (see `DEMO_SCRIPT.md` for exact commands)

1. `.venv/bin/pytest -q | tail` - show tests green.
2. Start emulator, receiver, sender with 3 paths; show per-path bytes.
3. Kill a path mid-transfer (CLI config `drop_after_s`, or Kill path button in UI) -> transfer still completes, hash matches.

4. Inject corruption -> show retransmissions counter, final hash identical.
5. Interrupt sender, rerun -> "resumed N chunks".
6. Open `results/*.png` and Wireshark with `mpft.lua` to show the 20-byte header decoded.
7. Web UI: two instances, Receiver ID, Accept, live chunk map, Download button.

---

11. How to split the study between three people

Member	Focus	Must be able to explain
Saksham	Protocol, sender, receiver, retries, resume (Parts 3, 5, 8)	`sender.py` `worker/requeue`, `receiver.py` `_store/finalize`, the header and message types
Parth	Emulator, experiments and graphs (Part 6)	`emulator.py` `_reader/_writer`, how each experiment is set up and why the numbers look that way
Anupria	Wireshark, dashboard and web layer, test report (Part 7, section below)	`mpft.lua`, the screenshots in `TEST_REPORT.md` section 6, `web/` event hub and UI

The split matches `CONTRIBUTIONS.md`. Everyone: know Part 1, the Part 2 diagram, the numbers in Part 6, and the limitations list.

12. Last-minute checklist

- [ ] Can draw the architecture and the message sequence from memory
- [ ] Can recite the 8 message types and the header fields in order
- [ ] Know the retry numbers (5 / 6 / 5 s) and why retries are prioritised
- [ ] Know E1, E2, E4 numbers
- [ ] Have run the demo twice end to end
- [ ] Know the limitations and the "future work" list

Added in the final update: views, scenarios, graph axis

- **Sender vs receiver page.** Same session, role-aware banner (`web/static/hero.js`): the sender reads "Delivered" or "Not delivered"; the receiver reads "Saved and verified" with Download, or "Nothing was saved". Panel order differs: receiver leads with verification.
- **Eight scenarios** (`web/netprofiles.py`): none, wan, asymmetric, lossy, heavy_loss, path_failure, path_flap (emulator `restore_after_s`), total_failure. Viva line: "total failure shows that no partial file is ever offered, because the receiver renames only after the whole-file SHA-256 matches."
- **Graph axis.** A 100 ms transfer cannot fill a seconds axis, so throughput is bucketed from chunk completions at duration/20 and sub-2 s transfers use milliseconds. Use the slow-WAN scenario to get readable curves.

Wireshark: what the screenshots show (`TEST_REPORT.md` section 6, `WIRESHARK_EVIDENCE.md`)

Capture on `lo0` of a 60 MB transfer over 3 emulated paths, path 3 killed at about 6 s.

- **Handshakes:** three SYN/SYN-ACK pairs to ports 9201-9203 (one per path) and three to 9000 (proxy to receiver). Later SYNs to 9203 are reconnect attempts after the kill, answered with RST (refused).
- **Conversations:** one TCP conversation per path; byte counts differ (fast path carries most). Each chunk crosses two hops, so everything appears twice.
- **Retransmissions:** `tcp.analysis.retransmission` is empty on loopback. MPFT's chunk retransmissions are not TCP retransmissions.
- **I/O graph:** one line per destination port; the 9203 line drops to zero at 6 s.
- **Decoded MANIFEST/CHUNK:** magic MP, version 1, type, session id, chunk index, payload length (262144 for a chunk), header CRC32, then JSON or raw bytes.
- **Path failure:** FIN/ACK on 9203 at 6.0 s.

Likely question: "*Why does the same chunk show up twice?*" Because the emulator proxy terminates each TCP connection and opens a new one to the receiver.